

HabitFlow

Product Requirements Document

Versão	1.0
Status	Em desenvolvimento
Data	01 de June de 2026
Stack	Fastify · Prisma · SQLite · Next.js 14
Tipo	Projeto de estudo — fullstack CRUD

1. Visão geral do produto

HabitFlow é um rastreador de hábitos diários construído como projeto fullstack de revisão técnica. O usuário cadastra hábitos que deseja cultivar — como "Ler 30 min" ou "Beber 2L de água" — e registra a conclusão de cada um ao longo do dia. A aplicação expõe uma API REST completa consumida por um frontend em Next.js.

O objetivo principal do projeto é cobrir os fundamentos de uma aplicação web real: modelagem de dados relacional, API REST estruturada com validação, consumo de API no frontend com gerenciamento de estado, e separação clara de responsabilidades entre camadas.

2. Objetivos técnicos

O projeto foi desenhado para revisar e consolidar os seguintes conceitos:

- Estruturar uma API REST com Fastify — rotas, plugins, schemas de validação com Zod
- Modelar banco de dados relacional com Prisma e SQLite — migrations, relações, constraints
- Implementar uma service layer na API, isolando lógica de negócio das rotas
- Consumir uma API REST no Next.js 14 com App Router — fetch server-side e client-side
- Gerenciar estado no frontend para feedback imediato ao usuário (marcar/desmarcar hábito)
- Configurar CORS, variáveis de ambiente e estrutura de monorepo com apps/api e apps/web

3. Funcionalidades

3.1 Gerenciamento de hábitos (CRUD)

- Criar hábito com nome definido pelo usuário
- Listar todos os hábitos com status de conclusão do dia atual
- Editar nome de um hábito existente via PATCH — sem afetar o histórico
- Deletar hábito e todos os seus logs via cascade delete configurado no Prisma

3.2 Registro diário de conclusão

- Marcar hábito como concluído hoje — cria um HabitLog com habitId e data atual

- Desmarcar hábito — remove o HabitLog do dia atual preservando histórico de dias anteriores
- Prevenção de duplicatas — constraint única no banco para (habitId, date); API retorna 409 em caso de conflito

3.3 Resumo do dia

- Endpoint GET /summary/today retornando: total de hábitos, quantidade concluída e percentual de conclusão
- Exibido no topo do frontend como painel de status do dia

4. Modelo de dados

4.1 Modelo Habit

Campo	Tipo	Descrição
id	String	Identificador único — gerado com cuid()
name	String	Nome do hábito definido pelo usuário
createdAt	DateTime	Timestamp de criação — gerado automaticamente
logs	HabitLog[]	Relação 1:N com os registros diários

4.2 Modelo HabitLog

Campo	Tipo	Descrição
id	String	Identificador único — gerado com cuid()
habitId	String	Chave estrangeira referenciando Habit.id
date	String	Data no formato YYYY-MM-DD (ex: 2025-06-01)
habit	Habit	Relação N:1 com o modelo Habit

4.3 Constraint de unicidade

Para garantir que um hábito não seja marcado duas vezes no mesmo dia, o schema Prisma aplica:

```
@@unique([habitId, date])
```

Se o frontend tentar criar um segundo log para o mesmo hábito no mesmo dia, a API retorna HTTP 409 Conflict.

5. Especificação da API

URL base (desenvolvimento): http://localhost:3333

5.1 Endpoints

Método	Rota	Descrição	Status
GET	/habits	Lista todos os hábitos com completedToday	200

POST	/habits	Cria novo hábito. Body: { name: string }	201
PATCH	/habits/:id	Atualiza nome do hábito. Body: { name: string }	200
DELETE	/habits/:id	Remove hábito e todos os seus logs	204
POST	/habits/:id/log	Marca hábito como feito hoje	201 / 409
DELETE	/habits/:id/log	Desmarca hábito do dia atual	204
GET	/summary/today	Retorna totais e percentual de conclusao do dia	200

5.2 Respostas de erro

400 Bad Request	Body invalido — validacao Zod falhou
404 Not Found	Habito nao encontrado pelo id informado
409 Conflict	Log duplicado — habito ja marcado hoje
500 Server Error	Erro interno nao tratado

6. Arquitetura e stack

Estrutura	Monorepo — apps/api e apps/web na mesma raiz
API	Fastify com TypeScript — rotas organizadas em modules/habits/
Validacao	Zod — schemas nos requests e responses
ORM	Prisma — migrations, queries e type safety
Banco	SQLite (arquivo local dev.db) — zero configuracao
Frontend	Next.js 14 com App Router e TypeScript
Estilizacao	Tailwind CSS
HTTP client	fetch nativo — sem axios ou bibliotecas externas

7. Fluxos principais

7.1 Abrir o app

- Frontend faz GET /habits ao montar a pagina
- API retorna array de habitos, cada um com completedToday: boolean
- Frontend renderiza a lista com checkboxes refletindo o estado do dia

7.2 Marcar habito como feito

- Usuario clica no checkbox de um habito nao concluido
- Frontend envia POST /habits/:id/log
- API valida, cria HabitLog no banco e retorna 201

- Frontend atualiza o estado local — sem novo GET
- Se habito ja estava marcado, API retorna 409 e frontend exibe feedback de erro

7.3 Criar novo habito

- Usuario preenche o input e clica em Adicionar
- Frontend envia POST /habits com body { name }
- API valida com Zod, salva no banco, retorna habito criado com 201
- Frontend insere o novo habito na lista com completedToday: false

8. Roadmap de implementacao

Dia	Entrega	Detalhes
Dia 1	Setup do projeto	Criar estrutura monorepo, configurar Fastify, Prisma e SQLite, rodar primeira migration
Dia 2	CRUD de habitos	Implementar as 4 rotas de habitos com validacao Zod e service layer
Dia 3	Logs diarios	Implementar POST e DELETE /habits/:id/log com constraint de unicidade
Dia 4	Frontend — lista	Pagina principal consumindo GET /habits, checkboxes e acoes de criar/deletar
Dia 5	Polimento	Loading states, feedback de erro, resumo do dia, UX minima funcional

9. Fora do escopo (v1)

- Autenticacao e multiusuario — sem login nesta versao
- Persistencia em nuvem — banco local SQLite apenas
- Notificacoes ou lembretes
- Visualizacao de historico de dias anteriores no frontend
- Testes automatizados — fora do objetivo desta revisao tecnica